FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

MATHÉMATIQUES & INFORMATIQUE

PROJET DevSecOps

Infrastructure Cloud AWS · CI/CD · Sécurité Intégrée

myway app

Yassine Farih - Adnane Dahbi - Anass Dabibe - Youssef Sarraf

Encadrant : Pr. Fouzia Benabbou

Projet DevSecOps Myway · Master d'Excellence en IA 2025–2026

AWS · Terraform · Ansible

Docker · Kubernetes · CI/CD

SonarCloud · Trivy · OWASP ZAP

Grafana · Prometheus · Monitoring

PLAN DE PRÉSENTATION

Sommaire

01 Application Myway + Architecture

02 AWS + Terraform (IaC)

03 Ansible + Docker

04 Kubernetes — Orchestration

05 Pipeline CI/CD

06 DevSecOps — Sécurité

07 Monitoring — Observabilité

08 Résultats & Avantages

09 Conclusion

APPLICATION MYWAY + ARCHITECTURE GLOBALE

Stack React + Node.js + SQLite · Pipeline DevSecOps de bout en bout sur AWS

Stack Technique

Frontend
React + Vite

Backend
Node.js + Express

Database
SQLite

Container
Docker

Orchestration
Kubernetes

CI/CD
GitHub Actions

Modules

Todos

Pomodoro

Ressources

Analytics

IA Chat

Architecture & Pipeline

Dev	CI/CD	Docker	Security	K8s
git push	Actions	Build	Scan	Deploy

AWS eu-west-1

Master 54.154.200.84

Worker 63.34.10.107

t3.micro × 2 · Kubernetes

Terraform

Ansible

SonarCloud

Trivy

OWASP ZAP

Prometheus

Grafana

Kubernetes

AWS — Infrastructure as Code (Terraform) + IAM

Ressources provisionnées automatiquement · Accès sécurisé avec principe du moindre privilège

Terraform — Ressources AWS

aws_key_pair
Clé SSH

aws_vpc
10.0.0.0/16

aws_subnet
10.0.1.0/24

aws_security_group
Ports ouverts

aws_instance ×2
k8s-master + worker

init → plan → apply → destroy

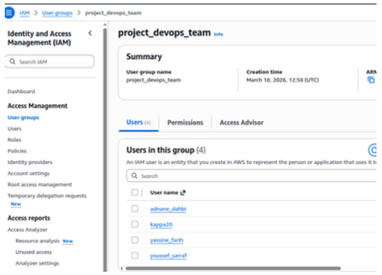
IAM — project_devops_team

Groupe project_devops_team
4 membres — accès console AWS + CLI

Politique IAM
AmazonEC2ReadOnlyAccess (moindre privilège)

```
(python_3_11_13) kappa@kappa-vivo:~/Documents,
terraform output

grafana_url = "http://[redacted].222:30030"
master_instance_id = "i-0d98a41f0bd49d062"
master_private_ip = "10.0.1.106"
master_public_ip = "[redacted].81"
myway_backend_url = "http://[redacted].222:30
myway_frontend_url = "http://[redacted].222:3
prometheus_url = "http://[redacted]:30090
ssh_command_master = "ssh -i ~/.ssh/id_rsa ec
ssh_command_worker = "ssh -i ~/.ssh/id_rsa ec
worker_instance_id = "i-0435992b869896d0c"
worker_private_ip = "10.0.1.172"
worker_public_ip = "[redacted].222"
(python_3_11_13) kappa@kappa-vivo:~/Documents,
```



Groupe IAM créé *AWS CLI configuré*

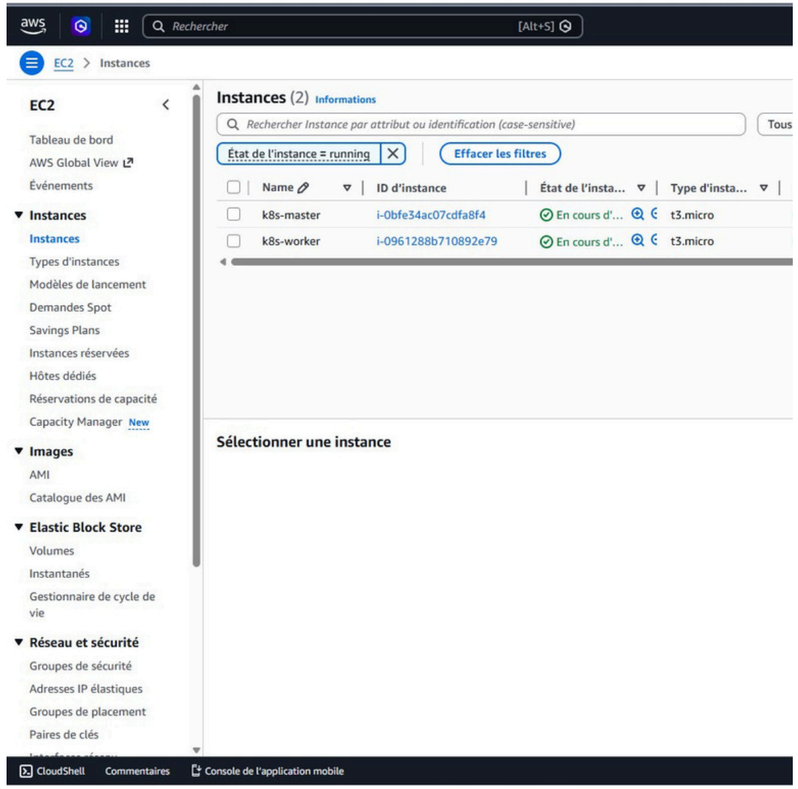
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project/Myway$
aws configure set aws_access_key_id [redacted]
aws configure set aws_secret_access_key [redacted]
aws configure set region eu-west-1
aws configure set output json
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project/Myway$
aws sts get-caller-identity
{
    "UserId": "4[redacted]7959",
    "Account": "4[redacted]7959",
    "Arn": "arn:aws:iam::41[redacted]9:root"
}
```

terraform output — IPs EC2 créées

AWS EC2 Console

k8s-master + k8s-worker Running (eu-west-1)



2 instances t3.micro · eu-west-1a · 3/3 vérifications réussies

ANSIBLE + DOCKER — Config & Conteneurisation

6 playbooks automatisés · Multi-stage build · Images ~25 MB · Push DockerHub avec tag SHA

ANSIBLE — 6 Playbooks

01 — Install Docker
Installation Docker sur nodes

02 — Install Kubernetes
kubeadm, kubelet, kubectl

03 — Init Master
kubeadm init — Master Ready

04 — Join Worker
kubectl get nodes — Master & Worker Ready

v1.29.0 — Master & Worker Ready

05 — Install Monitoring
Prometheus + Grafana sur cluster

06 — Install Dashboard
Kubernetes Dashboard — kubectl proxy

DOCKER — Multi-stage Build

Stage 1 — Build React

```
FROM node:18-alpine
WORKDIR /app
RUN npm install && npm run build
```

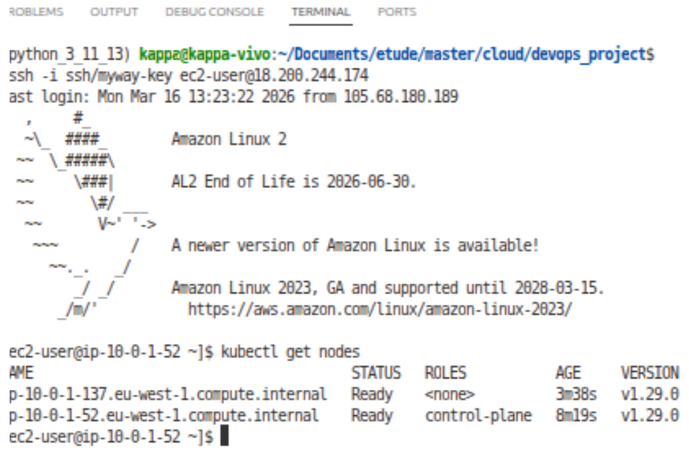
Stage 2 — nginx:alpine

```
FROM nginx:alpine
COPY --from=build /app/dist
/usr/share/nginx/html EXPOSE 80
```

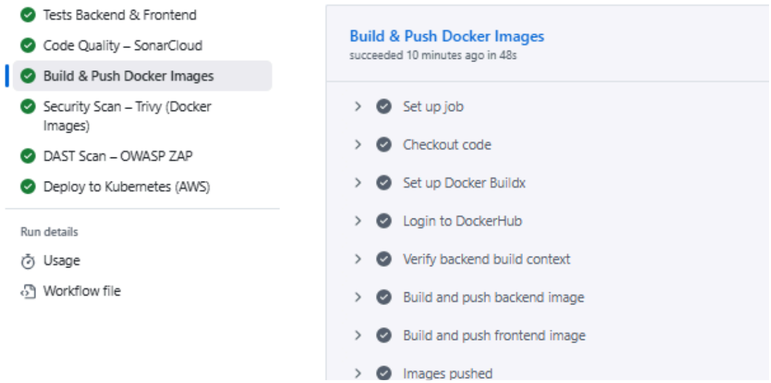
450 MB → 25 MB (-94%)

Push DockerHub avec tag :latest

Résultats & Screenshots



Ansible — terraform init, plan, apply



Docker — Job 3 Build & Push DockerHub SHA

KUBERNETES — Orchestration & Cluster

Cluster 2 nœuds · Rolling Updates · Health Probes · HPA · Zero Downtime

Concepts & Configuration

Rolling Update

Rolling Update par défaut Kubernetes

Health Probes

Ready Replicas vérifiées par le pipeline

Resource Limits

CPU: 100m–500m · RAM: 128Mi–512Mi

HPA Autoscaling

1–5 replicas · CPU trigger: 70%

Cluster — namespace myway

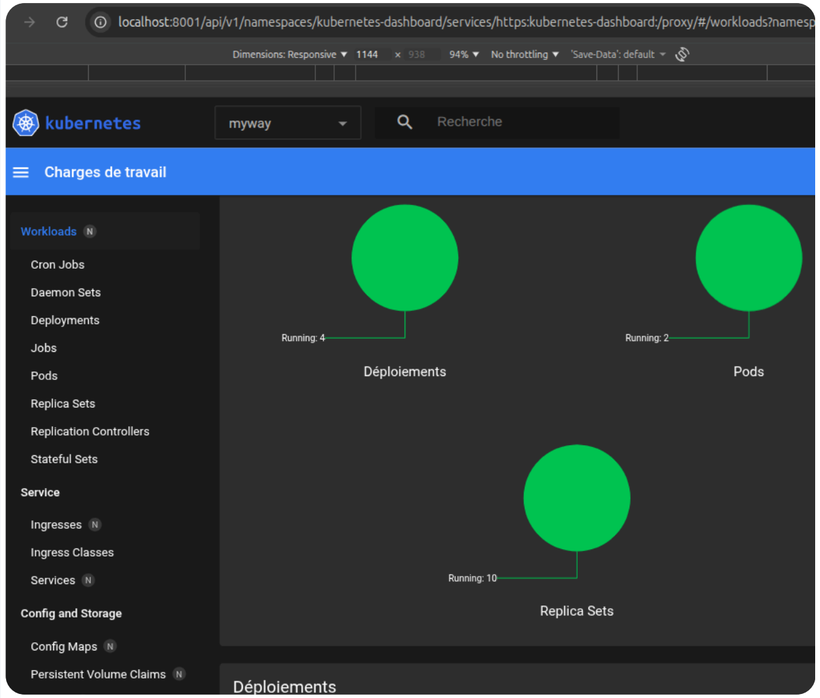
Master 54.154.200.84 — API · etcd · Scheduler

kubectl → port 6443

Worker 63.34.10.107

- **myway-backend**
  NodePort :30500
- **myway-frontend**
  NodePort :30080
- **prometheus**
  NodePort :30090
- **grafana**
  NodePort :30030

Kubernetes Dashboard



4 Deployments · 2 Pods · 10 ReplicaSets — Tout Running

PIPELINE CI/CD — GitHub Actions

6 jobs automatisés · Durée totale : 5 minutes 55 secondes

JOB 1 — 32s
Tests
npm test
build frontend

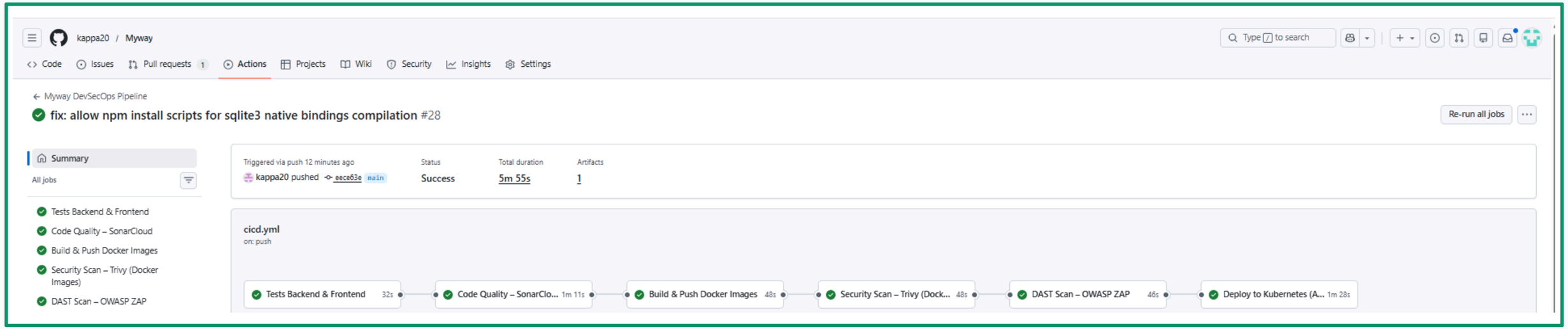
JOB 2 — 1m11s
SonarCloud
SAST
Quality Gate

JOB 3 — 48s
Docker Build
Build + Push
SHA tag

JOB 4 — 48s
Trivy Scan
CVE scan
SARIF report

JOB 5 — 46s
OWASP ZAP
DAST API
HTML report

JOB 6 — 1m28s
K8s Deploy
Rolling
+ Rollback



Pipeline complet — 6 jobs réussis — Total : 5m55s — github.com/kappa20/Myway

DevSecOps — Shift Left Security

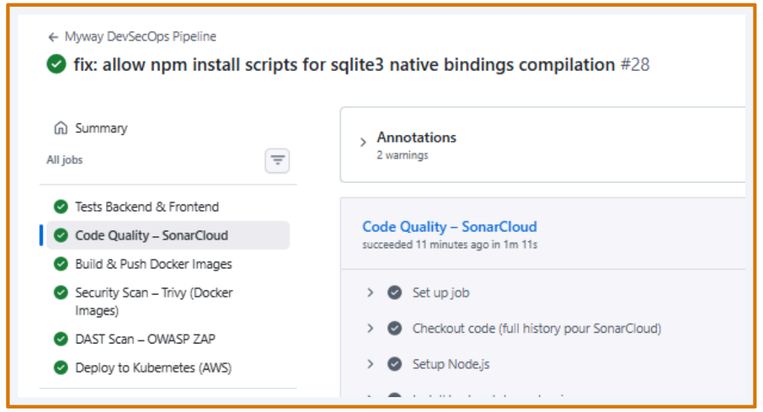
3 couches automatisées : SAST · Container Scan · DAST · à chaque commit

SonarCloud — SAST

Static Analysis

Analyse code sans exécuter

- Injections SQL
- XSS React/Express
- Code smells
- Security Hotspots

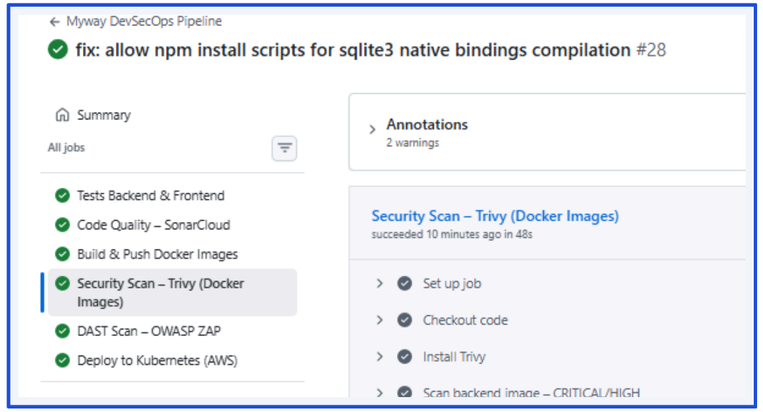


Trivy — Container

Image Scanner

Scanne les images Docker

- CVEs Alpine Linux
- Dépendances npm
- Secrets exposés
- Misconfigs Docker

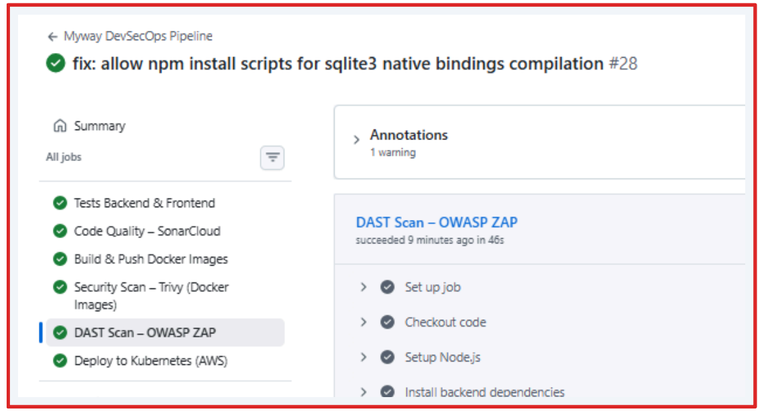


OWASP ZAP — DAST

Dynamic Testing

Teste l'API en exécution

- SQL Injection → détecté (audit)
- XSS Reflected → détecté (audit)
- CSP manquant → détecté (audit)
- Clickjacking → détecté (audit)



MONITORING — Prometheus & Grafana

Observabilité temps réel · CPU, RAM, Pods, Alertes · K8s Dashboard

Prometheus

Collecte des métriques

- Scrape auto via annotations K8s (prometheus.io/scrape)
- CPU, RAM, requêtes HTTP, pods running
- Alertes : Pod down > 1min · CPU > 80% · RAM > 85%

Grafana

Dashboards visuels

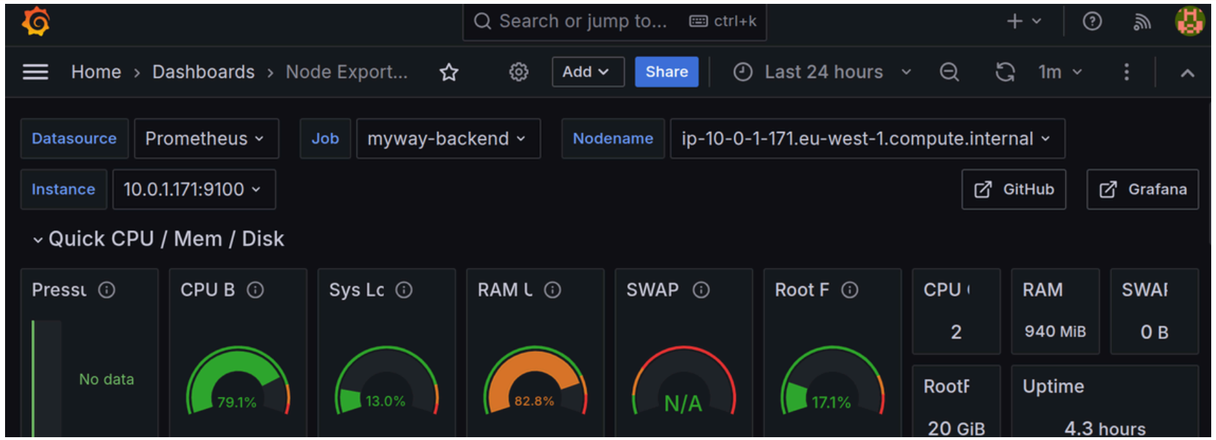
- Node Exporter : CPU 79.1%, RAM 82.8%
- Datasource Prometheus préconfigurée
- Port 30030 · Worker : 63.34.10.107

K8s Dashboard

Interface de gestion

- 4 Deployments · 2 Pods · 10 ReplicaSets
- Tous les workloads Running
- Namespace myway — vue graphique

Prometheus : process_cpu_seconds_total — port 30090



Grafana Node Exporter : CPU 79.1%, RAM 82.8%

RÉSULTATS & AVANTAGES

Ce pipeline DevSecOps apporte concrètement

5m55s
push → production

6
jobs DevSecOps

3
couches sécurité

0
downtime rolling

Automatisation totale
Infra → déploiement sans intervention manuelle

Scalabilité HPA
1→5 replicas automatique selon charge CPU

Observabilité
Prometheus + Grafana — alertes temps réel

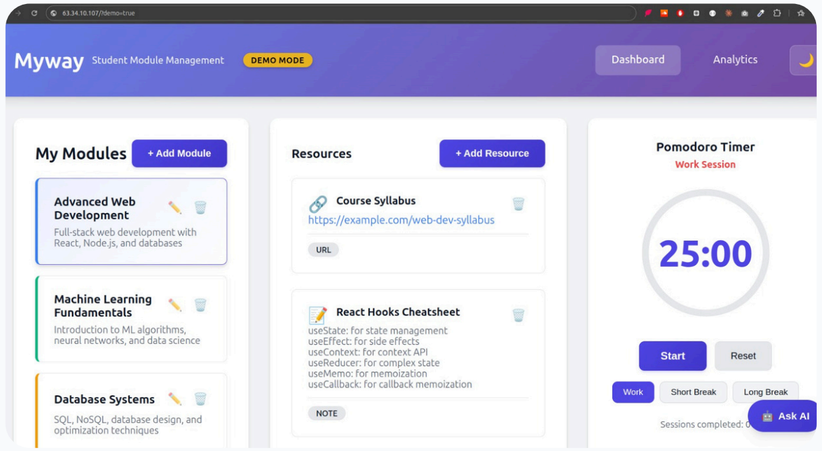
Traçabilité SHA
Chaque image liée au commit exact

Sécurité 3 couches
SonarCloud + Trivy + OWASP ZAP à chaque commit

Rolling Update
Zéro downtime · maxUnavailable: 0

Infrastructure as Code
Terraform versionné — reproductible en 30 min

Rollback auto
kubectl rollout undo si health checks échouent



Myway — App déployée sur Worker Node

MERCI !

Pour votre attention et votre évaluation

Des questions ?

Yassine Farih - Adnane Dahbi - Anass Dabibe - Youssef Sarraf

> *"Du code source à la production en moins de 6 minutes — avec zéro downtime et 3 couches de sécurité."*

Résumé du Projet

Infrastructure

AWS + Terraform + Ansible

Conteneurisation

Docker multi-stage + Kubernetes

CI/CD Pipeline

6 jobs · 5m55s · GitHub Actions

DevSecOps

Sonar + Trivy + OWASP ZAP

Monitoring

Prometheus + Grafana

Kubernetes

HPA · Rolling Update · Zéro downtime